

03-OpenAPI-AI-Services

1 OpenAPI for AI Services

Document Version: 1.0

Generated: December 4, 2025

Standard Version: OpenAPI 3.1

Status: Industry Standard for REST APIs

1.1 Table of Contents

1. [Overview](#)
 2. [Authoritative References](#)
 3. [Format Structure](#)
 4. [Procedural Use-Cases](#)
 5. [AI-Specific Patterns](#)
 6. [Examples](#)
 7. [Tools & Ecosystem](#)
 8. [Best Practices](#)
-

1.2 Overview

OpenAPI Specification (OAS), formerly known as Swagger, is a standard for defining REST API interfaces in a machine-readable format. For AI services, OpenAPI provides a structured way to document and implement ML model serving endpoints, inference APIs, and AI service integrations.

1.2.1 Key Features

- **Language-Agnostic:** Framework and language-independent API descriptions
- **Auto-Documentation:** Generate interactive API documentation automatically
- **Code Generation:** Create client SDKs and server stubs in multiple languages
- **Validation:** Request/response schema validation
- **Versioning:** Track API changes and maintain backward compatibility

1.2.2 Primary Use Cases for AI

1. **Model Serving APIs:** Document inference endpoints
 2. **ML Platform APIs:** Experiment tracking, model registry access
 3. **AI Service Integration:** Third-party AI service consumption
 4. **Batch Prediction APIs:** Async inference endpoints
 5. **Model Management:** CRUD operations for models
-

1.3 Authoritative References

1.3.1 Official Specifications

- **OpenAPI Specification:** <https://spec.openapis.org/oas/latest.html>
- **OpenAPI Initiative:** <https://www.openapis.org/>
- **GitHub Repository:** <https://github.com/OAI/OpenAPI-Specification>

1.3.2 Version History

- **OpenAPI 3.0** (2017): Major redesign from Swagger 2.0
- **OpenAPI 3.1** (2021): JSON Schema alignment, webhooks support

1.3.3 Standards Bodies

- **OpenAPI Initiative (OAI):** Under Linux Foundation

- **Contributors:** Google, Microsoft, IBM, Amazon, and others

1.4 Format Structure

OpenAPI specifications are typically written in **YAML** or **JSON**.

1.4.1 Core Components

```
graph TD
    A[OpenAPI Specification] --> B[Info]
    A --> C[Servers]
    A --> D[Paths]
    A --> E[Components]
    D --> F[Operations]
    F --> G[Parameters]
    F --> H[Request Body]
    F --> I[Responses]
    E --> J[Schemas]
    E --> K[Security Schemes]
```

1.4.2 1. Info Object

Role: Metadata about the API.

Key Fields: - **title**: API name - **version**: API version - **description**: Detailed API description - **contact**: Maintainer information - **license**: License details

Example:

```
info:
  title: Image Classification API
  version: 1.0.0
  description: Real-time image classification using deep learning models
  contact:
    name: AI Team
    email: ai-team@example.com
  license:
    name: Apache 2.0
    url: https://www.apache.org/licenses/LICENSE-2.0.html
```

1.4.3 2. Servers Object

Role: Defines API server URLs.

Example:

```
servers:
  - url: https://api.example.com/v1
    description: Production server
  - url: https://staging-api.example.com/v1
    description: Staging server
  - url: http://localhost:8000/v1
    description: Development server
```

1.4.4 3. Paths Object

Role: Defines available endpoints and operations.

Structure:

```

paths:
  /predict:
    post:
      summary: Make a prediction
      operationId: predict
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PredictionRequest'
      responses:
        '200':
          description: Successful prediction
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/PredictionResponse'

```

1.4.5 4. Components Object

Role: Reusable definitions (schemas, parameters, responses).

Schemas (data models):

```

components:
  schemas:
    PredictionRequest:
      type: object
      required:
        - image
      properties:
        image:
          type: string
          format: byte
          description: Base64-encoded image
        model_id:
          type: string
          description: Model identifier
          default: "resnet50"

    PredictionResponse:
      type: object
      properties:
        predictions:
          type: array
          items:
            $ref: '#/components/schemas/Prediction'
        inference_time_ms:
          type: number
          format: float

    Prediction:
      type: object
      properties:
        class_name:
          type: string
        class_id:
          type: integer
        confidence:
          type: number
          format: float
          minimum: 0
          maximum: 1

```

1.4.6 5. Security Schemes

Role: Authentication and authorization mechanisms.

Example:

```
components:
  securitySchemes:
    ApiKeyAuth:
      type: apiKey
      in: header
      name: X-API-Key

    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  security:
    - ApiKeyAuth: []
    - BearerAuth: []
```

1.5 Procedural Use-Cases

1.5.1 Use-Case 1: Designing an Image Classification API

Goal: Create a REST API for image classification with OpenAPI documentation.

Workflow:

```
graph LR
  A[Define Requirements] --> B[Design Endpoints]
  B --> C[Create OpenAPI Spec]
  C --> D[Generate Server Code]
  D --> E[Implement Logic]
  E --> F[Deploy API]
  F --> G[Generate Client SDKs]
```

Step-by-Step Procedure:

1. Define API Specification:

```
openapi: 3.1.0
info:
  title: Image Classification API
  version: 1.0.0
  description: |
    Deep learning-based image classification service.
    Supports multiple model architectures and real-time inference.

servers:
  - url: https://api.mlplatform.com/v1

paths:
  /models:
    get:
      summary: List available models
      operationId: listModels
      tags:
        - Models
      responses:
        '200':
          description: List of available models
          content:
```

```

    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/ModelInfo'

/classify:
  post:
    summary: Classify an image
    operationId: classifyImage
    tags:
      - Inference
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            properties:
              file:
                type: string
                format: binary
              model:
                type: string
                default: resnet50
              top_k:
                type: integer
                minimum: 1
                maximum: 10
                default: 5
            responses:
              '200':
                description: Classification results
                content:
                  application/json:
                    schema:
                      $ref: '#/components/schemas/ClassificationResult'
              '400':
                description: Invalid input
                content:
                  application/json:
                    schema:
                      $ref: '#/components/schemas/Error'
              '500':
                description: Internal server error
                content:
                  application/json:
                    schema:
                      $ref: '#/components/schemas/Error'
    security:
      - ApiKeyAuth: []

components:
  schemas:
    ModelInfo:
      type: object
      properties:
        id:
          type: string
          example: resnet50
        name:
          type: string
          example: ResNet-50
        description:
          type: string
        input_shape:
          type: array
          items:
            type: integer
          example: [224, 224, 3]
        classes:
          type: integer

```

```

    example: 1000

ClassificationResult:
  type: object
  properties:
    predictions:
      type: array
      items:
        type: object
        properties:
          class_name:
            type: string
            example: "golden_retriever"
          class_id:
            type: integer
            example: 207
          confidence:
            type: number
            format: float
            example: 0.9523
        model_used:
          type: string
          example: "resnet50"
      inference_time_ms:
        type: number
        format: float
        example: 45.2

Error:
  type: object
  properties:
    error:
      type: string
    message:
      type: string
    status_code:
      type: integer

securitySchemes:
  ApiKeyAuth:
    type: apiKey
    in: header
    name: X-API-Key

```

2. Generate Server Stub (using FastAPI):

```

from fastapi import FastAPI, File, UploadFile, HTTPException, Security
from fastapi.security.api_key import APIKeyHeader
from pydantic import BaseModel, Field
from typing import List
import time

app = FastAPI(
    title="Image Classification API",
    version="1.0.0",
    description="Deep learning-based image classification service"
)

# Security
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=True)

def verify_api_key(api_key: str = Security(api_key_header)):
    if api_key != "your-secret-key":
        raise HTTPException(status_code=403, detail="Invalid API Key")
    return api_key

# Models
class Prediction(BaseModel):
    class_name: str
    class_id: int

```

```

confidence: float = Field(ge=0, le=1)

class ClassificationResult(BaseModel):
    predictions: List[Prediction]
    model_used: str
    inference_time_ms: float

class ModelInfo(BaseModel):
    id: str
    name: str
    description: str
    input_shape: List[int]
    classes: int

# Endpoints
@app.get("/models", response_model=List[ModelInfo])
def list_models():
    return [
        ModelInfo(
            id="resnet50",
            name="ResNet-50",
            description="50-layer residual network",
            input_shape=[224, 224, 3],
            classes=1000
        )
    ]

@app.post("/classify", response_model=ClassificationResult)
async def classify_image(
    file: UploadFile = File(...),
    model: str = "resnet50",
    top_k: int = 5,
    api_key: str = Security.verify_api_key
):
    start_time = time.time()

    # Read image
    image_bytes = await file.read()

    # Run inference (placeholder)
    predictions = [
        Prediction(class_name="golden_retriever", class_id=207, confidence=0.9523),
        Prediction(class_name="labrador_retriever", class_id=208, confidence=0.0321),
    ]

    inference_time = (time.time() - start_time) * 1000

    return ClassificationResult(
        predictions=predictions[:top_k],
        model_used=model,
        inference_time_ms=inference_time
    )

```

3. Generate Client SDK (Python):

```

import requests
from typing import List, Dict

class ImageClassificationClient:
    def __init__(self, base_url: str, api_key: str):
        self.base_url = base_url
        self.headers = {"X-API-Key": api_key}

    def list_models(self) -> List[Dict]:
        """Get list of available models"""
        response = requests.get(
            f"{self.base_url}/models",
            headers=self.headers
        )
        response.raise_for_status()
        return response.json()

    def classify(self, image_path: str, model: str = "resnet50",
                top_k: int = 5) -> Dict:
        """Classify an image"""
        with open(image_path, 'rb') as f:
            files = {'file': f}
            params = {'model': model, 'top_k': top_k}

            response = requests.post(
                f"{self.base_url}/classify",
                headers=self.headers,
                files=files,
                params=params
            )

            response.raise_for_status()
            return response.json()

# Usage
client = ImageClassificationClient(
    "https://api.mlplatform.com/v1",
    "your-api-key"
)

# List models
models = client.list_models()
print(f"Available models: {[m['id'] for m in models]}")

# Classify image
result = client.classify("dog.jpg", model="resnet50", top_k=3)
print(f"Top prediction: {result['predictions'][0]['class_name']} "
      f"({result['predictions'][0]['confidence']:.2%})")

```

1.5.2 Use-Case 2: Text Generation API (LLM Service)

Goal: Design an API for text generation services.

OpenAPI Specification:

```

openapi: 3.1.0
info:
  title: Text Generation API
  version: 2.0.0
  description: Large Language Model text generation service

paths:
  /generate:
    post:
      summary: Generate text completion
      requestBody:
        required: true

```

```

content:
  application/json:
    schema:
      type: object
      required:
        - prompt
      properties:
        prompt:
          type: string
          description: Input text prompt
          example: "Write a poem about AI:"
        max_tokens:
          type: integer
          minimum: 1
          maximum: 2048
          default: 100
        temperature:
          type: number
          format: float
          minimum: 0
          maximum: 2
          default: 0.7
          description: Sampling temperature (higher = more random)
        top_p:
          type: number
          format: float
          minimum: 0
          maximum: 1
          default: 1.0
          description: Nucleus sampling threshold
        stop:
          type: array
          items:
            type: string
          description: Stop sequences
          example: ["\n", "END"]
responses:
  '200':
    description: Generated text
    content:
      application/json:
        schema:
          type: object
          properties:
            generated_text:
              type: string
            tokens_used:
              type: integer
            finish_reason:
              type: string
              enum: [length, stop, error]

/generate/stream:
  post:
    summary: Stream text generation
    requestBody:
      $ref: '#/paths/~1generate/post/requestBody'
    responses:
      '200':
        description: Server-sent events stream
        content:
          text/event-stream:
            schema:
              type: object
              properties:
                token:
                  type: string
                done:
                  type: boolean

```

Implementation (FastAPI with streaming):

```

from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
import asyncio
import json

app = FastAPI()

class GenerateRequest(BaseModel):
    prompt: str
    max_tokens: int = Field(100, ge=1, le=2048)
    temperature: float = Field(0.7, ge=0, le=2)
    top_p: float = Field(1.0, ge=0, le=1)
    stop: List[str] = Field(default_factory=list)

class GenerateResponse(BaseModel):
    generated_text: str
    tokens_used: int
    finish_reason: str

@app.post("/generate", response_model=GenerateResponse)
async def generate(request: GenerateRequest):
    # Simulated generation
    generated = f"{request.prompt} [Generated text with {request.max_tokens} tokens]"

    return GenerateResponse(
        generated_text=generated,
        tokens_used=len(generated.split()),
        finish_reason="length"
    )

@app.post("/generate/stream")
async def generate_stream(request: GenerateRequest):
    async def event_generator():
        tokens = ["This", " is", " a", " streamed", " response"]
        for token in tokens:
            await asyncio.sleep(0.1) # Simulate generation delay
            yield f"data: {json.dumps({'token': token, 'done': False})}\n\n"
        yield f"data: {json.dumps({'token': '', 'done': True})}\n\n"

    return StreamingResponse(
        event_generator(),
        media_type="text/event-stream"
    )

```

1.5.3 Use-Case 3: Batch Prediction API

Goal: Design an asynchronous batch prediction service.

Workflow:

```
sequenceDiagram
    participant Client
    participant API
    participant Queue
    participant Worker
    participant Storage

    Client->>API: POST /batch/jobs (upload data)
    API->>Queue: Enqueue job
    API->>Client: 202 Accepted (job_id)
    Client->>API: GET /batch/jobs/{job_id}
    API->>Client: 200 OK (status: pending)
    Worker->>Queue: Poll for jobs
    Queue->>Worker: Job data
    Worker->>Worker: Process batch
    Worker->>Storage: Save results
    Worker->>API: Update status
    Client->>API: GET /batch/jobs/{job_id}
    API->>Client: 200 OK (status: completed)
    Client->>API: GET /batch/jobs/{job_id}/results
    API->>Storage: Fetch results
    Storage->>API: Results data
    API->>Client: 200 OK (results)
```

OpenAPI Specification:

```
paths:
  /batch/jobs:
    post:
      summary: Create batch prediction job
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                input_data:
                  type: array
                  items:
                    type: object
                model_id:
                  type: string
      responses:
        '202':
          description: Job accepted
          content:
            application/json:
              schema:
                type: object
                properties:
                  job_id:
                    type: string
                    format: uuid
                  status:
                    type: string
                    enum: [pending, processing, completed, failed]

  /batch/jobs/{job_id}:
    get:
      summary: Get job status
      parameters:
        - name: job_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
```

```
'200':
  description: Job status
  content:
    application/json:
      schema:
        type: object
        properties:
          job_id:
            type: string
          status:
            type: string
          created_at:
            type: string
            format: date-time
          progress:
            type: number
            format: float
            minimum: 0
            maximum: 100

/batch/jobs/{job_id}/results:
  get:
    summary: Get job results
    parameters:
      - name: job_id
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Job results
        content:
          application/json:
            schema:
              type: object
              properties:
                results:
                  type: array
                  items:
                    type: object
```

1.6 AI-Specific Patterns

1.6.1 1. Model Version Management

```
paths:
  /models/{model_id}/versions:
    get:
      summary: List model versions
      parameters:
        - name: model_id
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: List of versions
          content:
            application/json:
              schema:
                type: array
                items:
                  type: object
                  properties:
                    version:
                      type: string
                    created_at:
                      type: string
                      format: date-time
                    metrics:
                      type: object
                      properties:
                        accuracy:
                          type: number
                        f1_score:
                          type: number
```

1.6.2 2. Explainability Endpoints

```
paths:
  /explain:
    post:
      summary: Get prediction explanation
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                input:
                  type: object
                prediction_id:
                  type: string
      responses:
        '200':
          description: Explanation
          content:
            application/json:
              schema:
                type: object
                properties:
                  feature_importance:
                    type: array
                  items:
                    type: object
                    properties:
                      feature:
                        type: string
                      importance:
                        type: number
```

1.6.3 3. Monitoring & Health Checks

```

paths:
  /health:
    get:
      summary: Health check
      responses:
        '200':
          description: Service healthy
          content:
            application/json:
              schema:
                type: object
                properties:
                  status:
                    type: string
                    enum: [healthy, degraded, unhealthy]
                  model_loaded:
                    type: boolean
                  gpu_available:
                    type: boolean

  /metrics:
    get:
      summary: Get service metrics
      responses:
        '200':
          description: Metrics
          content:
            application/json:
              schema:
                type: object
                properties:
                  requests_total:
                    type: integer
                  avg_inference_time_ms:
                    type: number
                  error_rate:
                    type: number

```

1.7 Examples

1.7.1 Complete API Specification (Real YAML)

```

openapi: 3.1.0
info:
  title: ML Platform API
  version: 1.0.0
  description: Comprehensive ML service API
  contact:
    name: AI Platform Team
    email: support@mlplatform.com
  license:
    name: MIT
servers:
  - url: https://api.mlplatform.com/v1
    description: Production
  - url: https://staging-api.mlplatform.com/v1
    description: Staging

security:
  - BearerAuth: []

paths:
  /predict:
    post:
      summary: Real-time prediction
      tags: [Inference]

```

```

    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PredictionRequest'
          example:
            model_id: "clf-v2"
            features:
              age: 35
              income: 75000
              score: 0.85
      responses:
        '200':
          description: Prediction result
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/PredictionResponse'
              example:
                prediction: 1
                confidence: 0.923
                model_version: "2.1.0"

components:
  schemas:
    PredictionRequest:
      type: object
      required:
        - features
      properties:
        model_id:
          type: string
        features:
          type: object
        additionalProperties: true

    PredictionResponse:
      type: object
      properties:
        prediction:
          oneOf:
            - type: number
            - type: string
            - type: array
        confidence:
          type: number
          format: float
        model_version:
          type: string

  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

```

1.8 Tools & Ecosystem

1.8.1 Documentation & Validation

Tool	Description	Homepage
Swagger UI	Interactive API documentation	https://swagger.io/tools/swagger-ui/
ReDoc	Beautiful API docs	https://redocly.com/redoc/
Stoplight	API design platform	https://stoplight.io/
Spectral	OpenAPI linter	https://stoplight.io/open-source/spectral

1.8.2 Code Generation

Tool	Description	Homepage
OpenAPI Generator	Multi-language client/server generation	https://openapi-generator.tech/
Swagger Codegen	Code generation tool	https://swagger.io/tools/swagger-codegen/
FastAPI	Python framework with auto OpenAPI	https://fastapi.tiangolo.com/

1.8.3 Testing & Mocking

Tool	Description	Homepage
Postman	API testing platform	https://www.postman.com/
Prism	Mock server from OpenAPI	https://stoplight.io/open-source/prism
Dredd	API testing tool	https://dredd.org/

1.9 Best Practices

1.9.1 1. Versioning

```
# Include version in URL
servers:
  - url: https://api.example.com/v1

# Document breaking changes
info:
  version: 2.0.0
  x-api-lifecycle:
    deprecated: false
    sunset: "2026-01-01"
```

1.9.2 2. Error Responses

```
components:
  schemas:
    Error:
      type: object
      required:
        - error
        - message
      properties:
        error:
          type: string
          enum: [validation_error, not_found, internal_error]
        message:
          type: string
        details:
          type: object
```

1.9.3 3. Pagination

```
parameters:
  - name: page
    in: query
    schema:
      type: integer
      minimum: 1
      default: 1
  - name: per_page
    in: query
    schema:
      type: integer
      minimum: 1
      maximum: 100
      default: 20
```

Navigation: [Back to Index](#) | [Previous: ONNX](#) | [Next: Model Cards](#)